

Relatório - Exercício Prático 01

Laboratório de Arquitetura de Computadores II

Aluno(a): [PREENCHER NOME]

Matrícula: [PREENCHER MATRÍCULA]

Turma/Grupo: [PREENCHER]

Este é um rascunho editável por nova exportação: revise nome, matrícula, datas e qualquer orientação específica do professor antes da submissão.

Parte 1 - Somadores no Logisim

Foram construídos um meio somador, um somador completo de 1 bit e um somador de 4 bits com propagação de carry, usando subcircuitos.

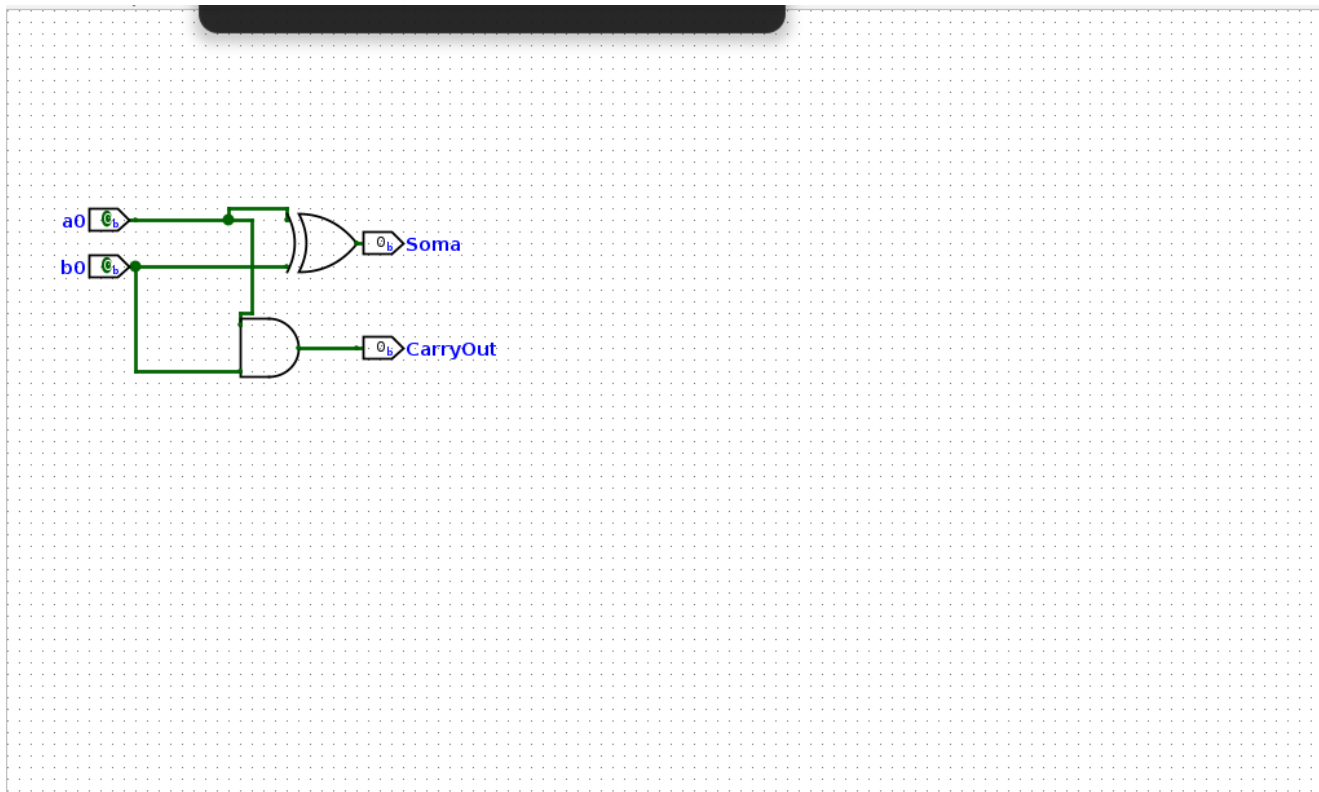


Figura 1 - Meio somador no Logisim.

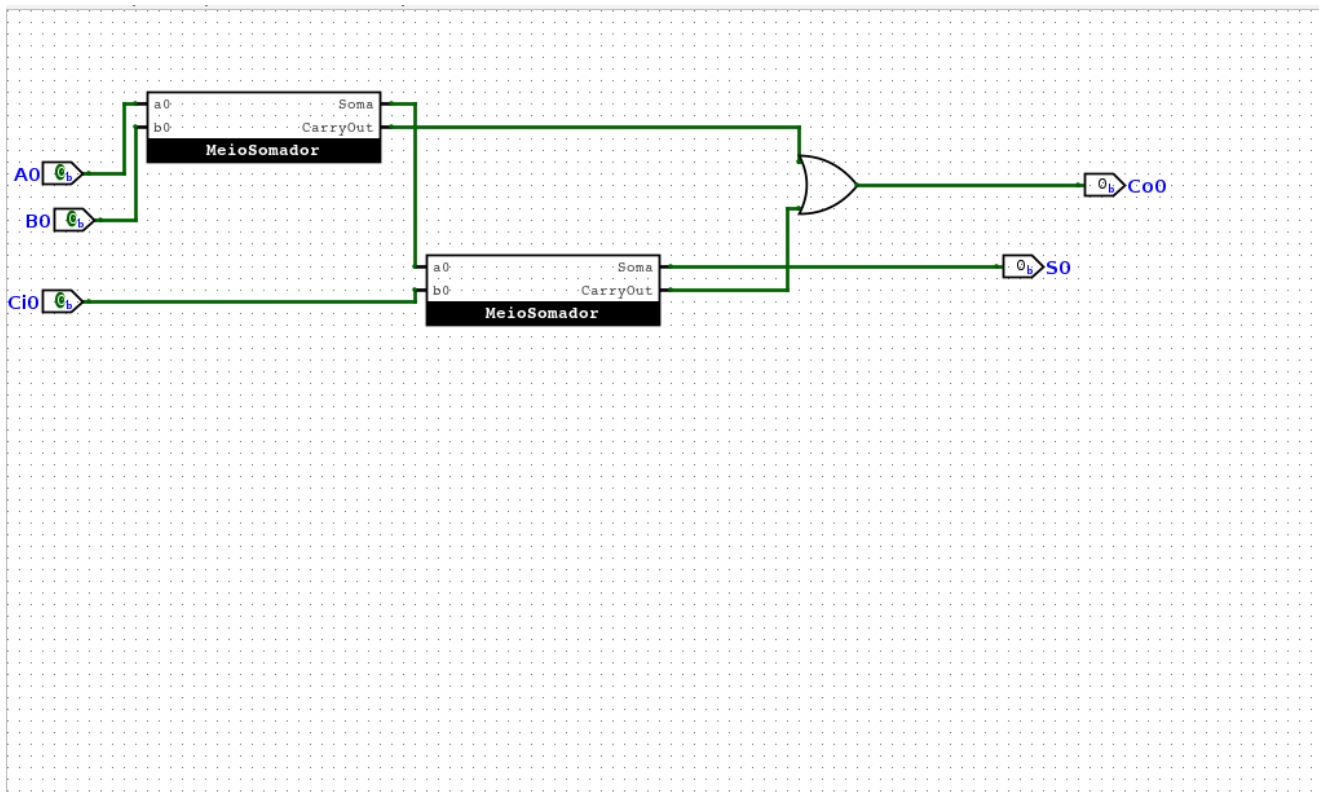


Figura 2 - Somador completo de 1 bit no Logisim.

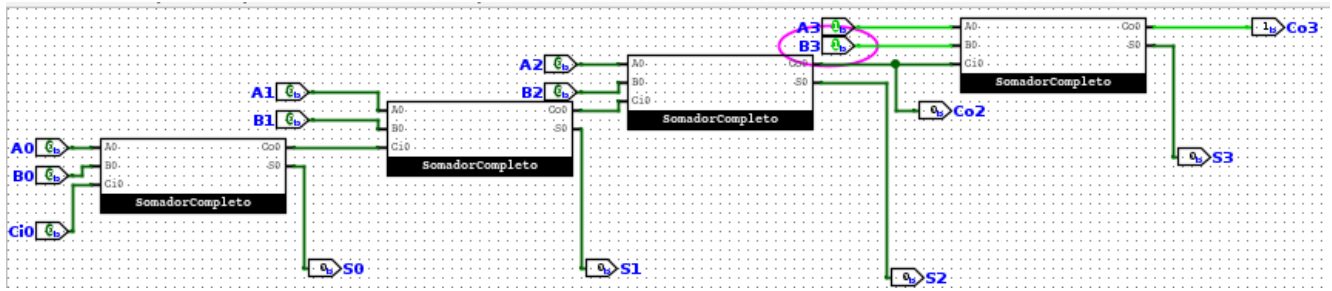


Figura 3 - Estrutura interna do somador de 4 bits.

Análise de tempo

Questão	Resposta
1. Problema de tempo	O carry de uma posição precisa se propagar para a seguinte. Em um somador ripple-carry, essa dependência cresce linearmente com o número de bits.
2. Soma e carry em 4 bits	Considerando portas de 10 ns e o somador completo construído com dois meio-somadores e uma OR: o carry final tem pior caso de aproximadamente 90 ns; o bit S3 estabiliza em aproximadamente 80 ns. A estimativa considera 30 ns no primeiro estágio e mais 20 ns por estágio de propagação.
3. Somador de 32 bits	Seriam necessários 32 somadores completos encadeados, e o atraso de carry se tornaria significativo.
4. Frequência em 32 bits	Pelo mesmo modelo, o carry final tem aproximadamente 650 ns (30 ns + 31 x 20 ns). Portanto, a frequência máxima é aproximadamente $1/650 \text{ ns} = 1,54 \text{ MHz}$.
5. Como acelerar	Pode-se usar um somador carry look-ahead, carry select ou carry skip para reduzir a dependência sequencial do carry.

Parte 2 - Calculadora de 4 bits

A calculadora recebe duas parcelas de 4 bits, exibe cada parcela e a soma em hexadecimal nos displays de 7 segmentos e possui indicação de overflow.

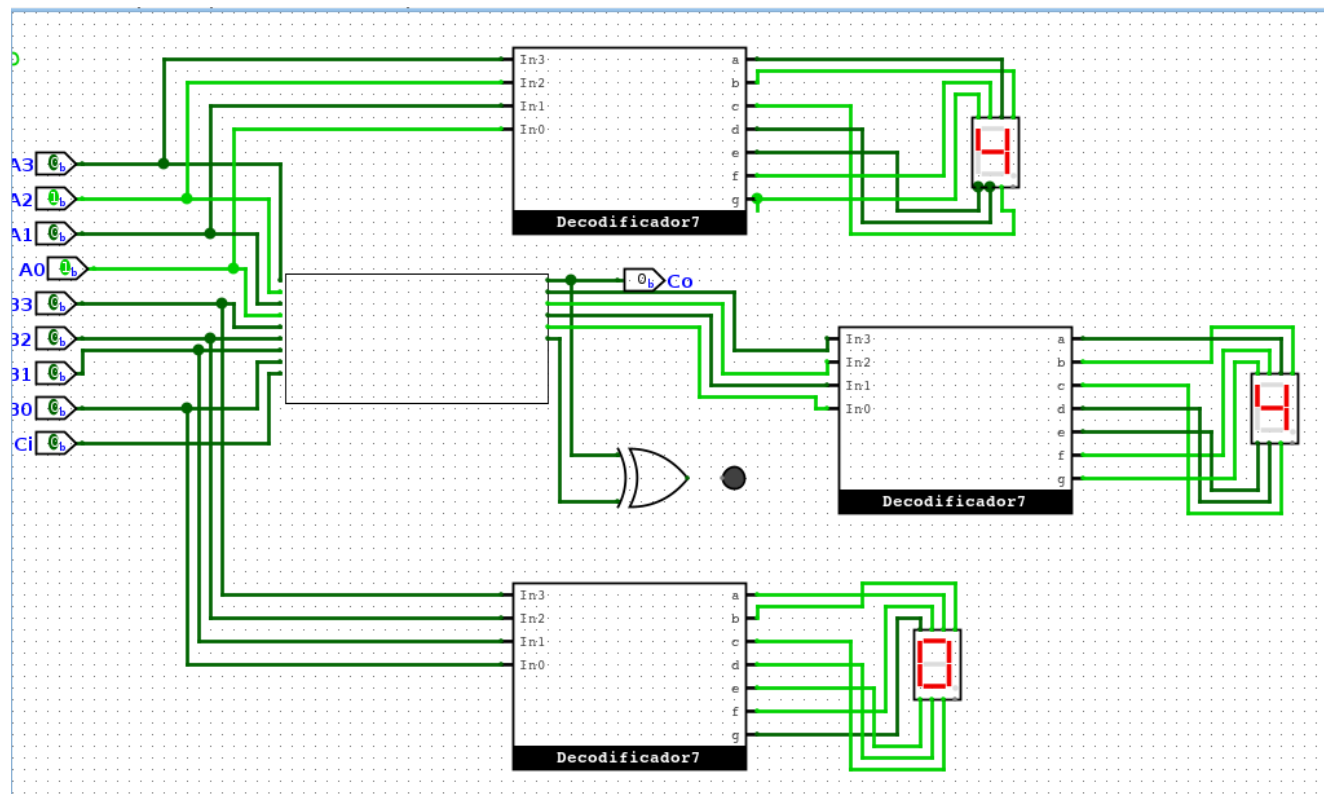


Figura 4 - Circuito principal com os displays: $5 + A = F$.

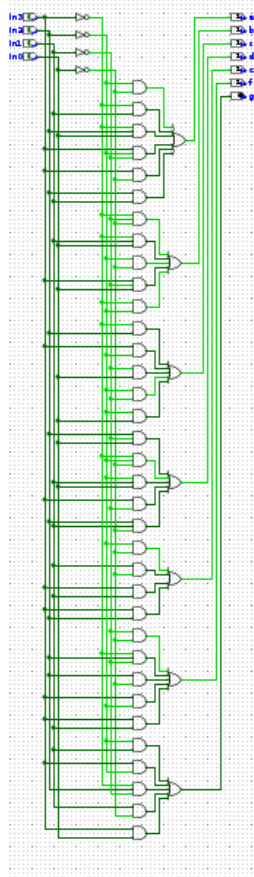


Figura 5 - Estrutura interna do decodificador para display de 7 segmentos.

Parte 3 - Soma com complemento de dois e overflow

O overflow assinado é identificado por $Co2 \text{ XOR } Co3$, isto é, pela diferença entre o carry que entra no bit de sinal e o carry que sai dele.

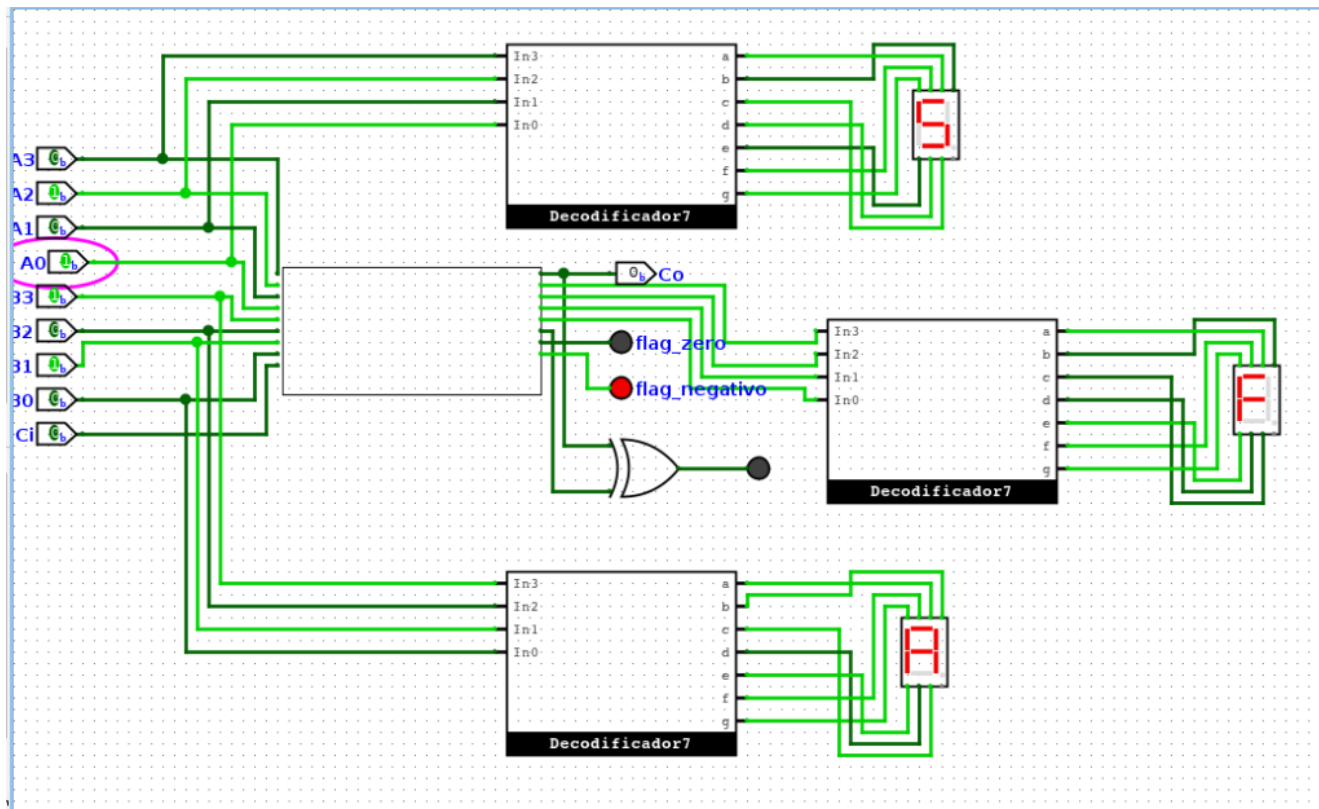


Figura 6 - Operação a: +5 + (-6).

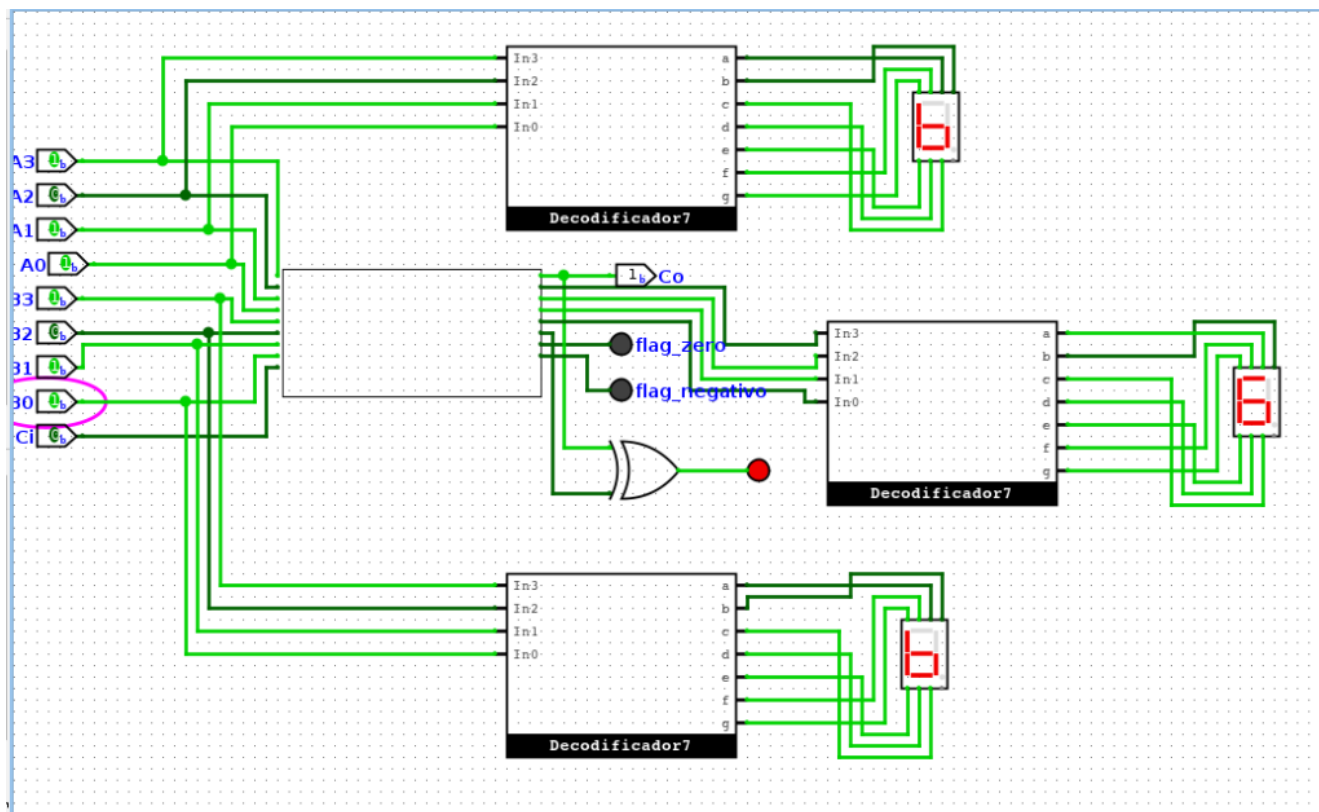


Figura 7 - Operação b: -5 + (-5).

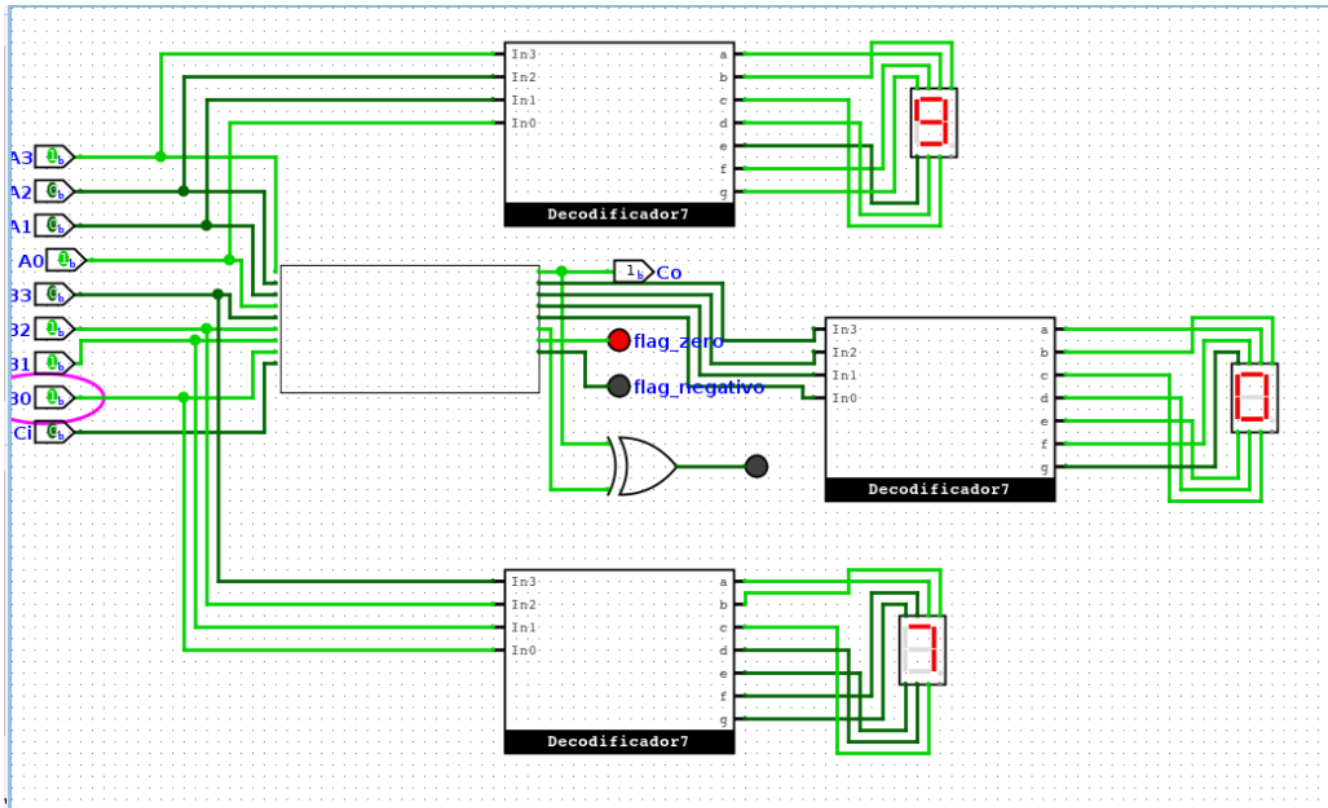


Figura 8 - Operação c: $-7 + (-9)$, conforme a codificação inserida.

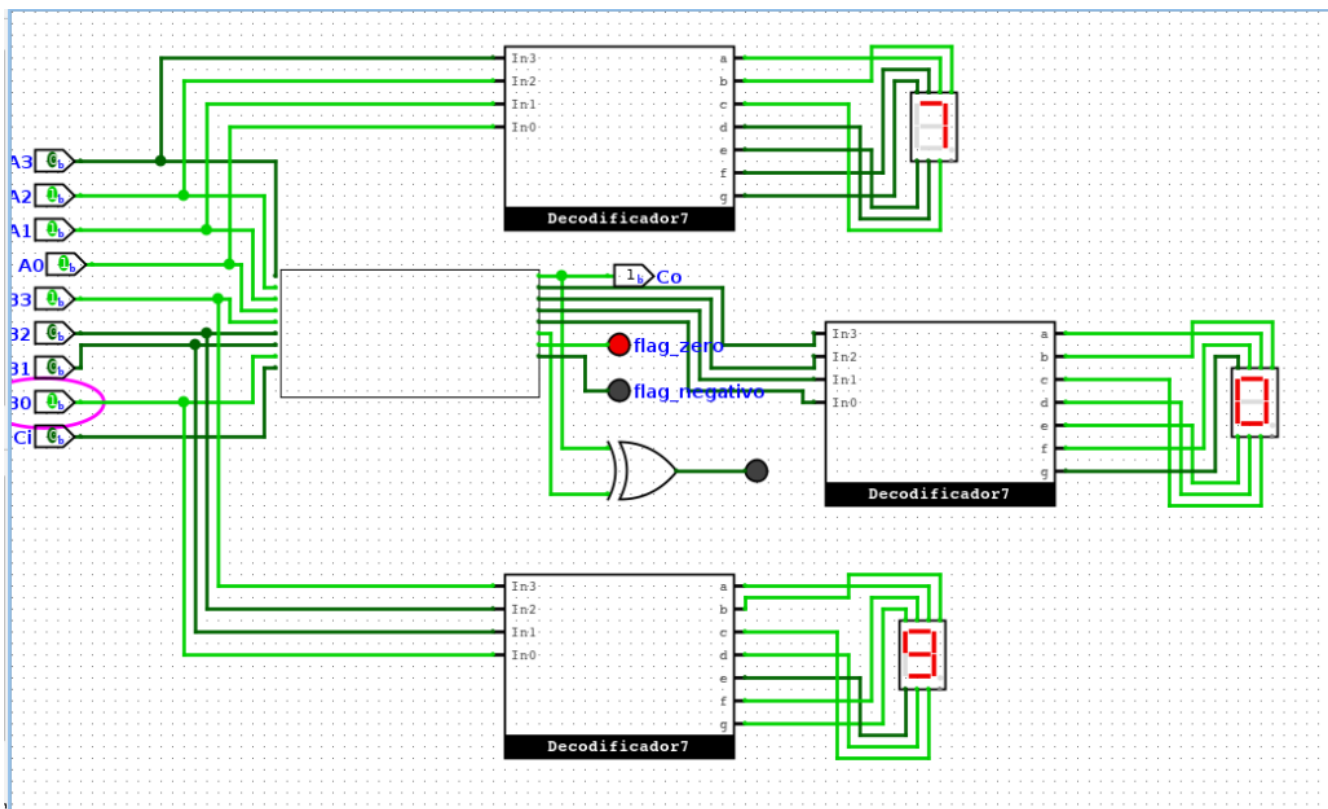


Figura 9 - Operação d: $+7 + +9$, conforme a codificação inserida.

Resultados e observação

Questão	Resposta
a) $+5 + (-6)$	Resultado em 4 bits: F (1111), equivalente a -1. Não há overflow assinado.
b) $-5 + (-5)$	Resultado em 4 bits: 6 (0110). Há overflow assinado, pois -10 não cabe no intervalo de 4 bits com sinal.
c) $-7 + (-9)$	Em complemento de dois com 4 bits, -9 não é representável (o intervalo é de -8 a +7). A leitura do padrão truncado deve ser validada com o professor.
d) $+7 + +9$	Em complemento de dois com 4 bits, +9 também não é representável. O padrão 1001 é interpretado como -7; por isso a XOR pode permanecer em 0.
Porta utilizada	XOR. Ela gera 1 somente quando Co2 e Co3 são diferentes, indicando overflow em soma com sinal.

Parte 4 - Flags

A flag Zero é uma NOR de quatro entradas: $\text{Zero} = \text{NOT}(\text{S3 OR S2 OR S1 OR S0})$. A flag Negativo é ligada diretamente ao bit mais significativo S3.

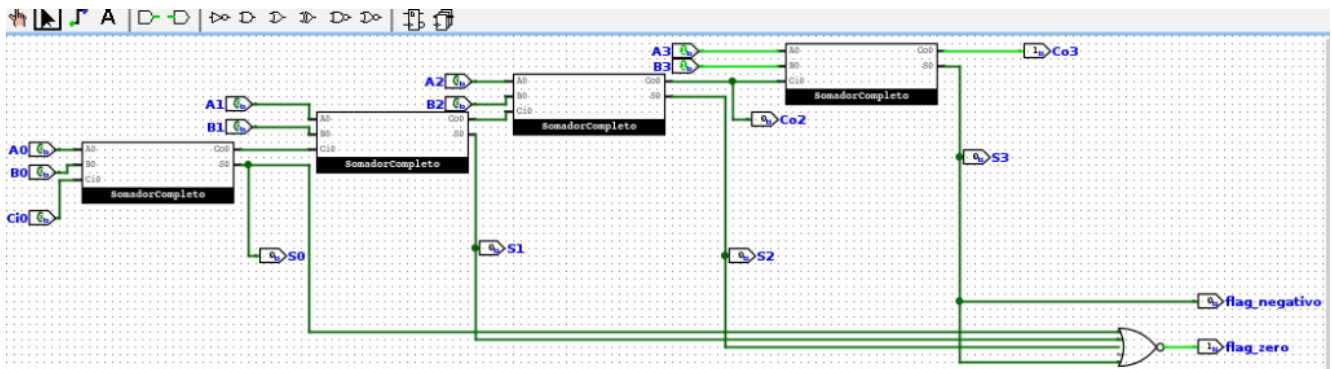


Figura 10 - Implementação interna das flags Zero e Negativo.

Aplicações: a flag Zero pode ser usada por instruções de comparação e desvio condicional (igualdade); a flag Negativo pode ser usada para testar o sinal de um resultado em aritmética com complemento de dois.

Parte 5 - Somador completo no Tinkercad

O circuito utiliza portas AND, OR e XOR para implementar um somador completo de 1 bit. Para $a = 1$, $b = 1$ e $\text{carryin} = 1$, espera-se $\text{Soma} = 1$ e $\text{Carryout} = 1$.

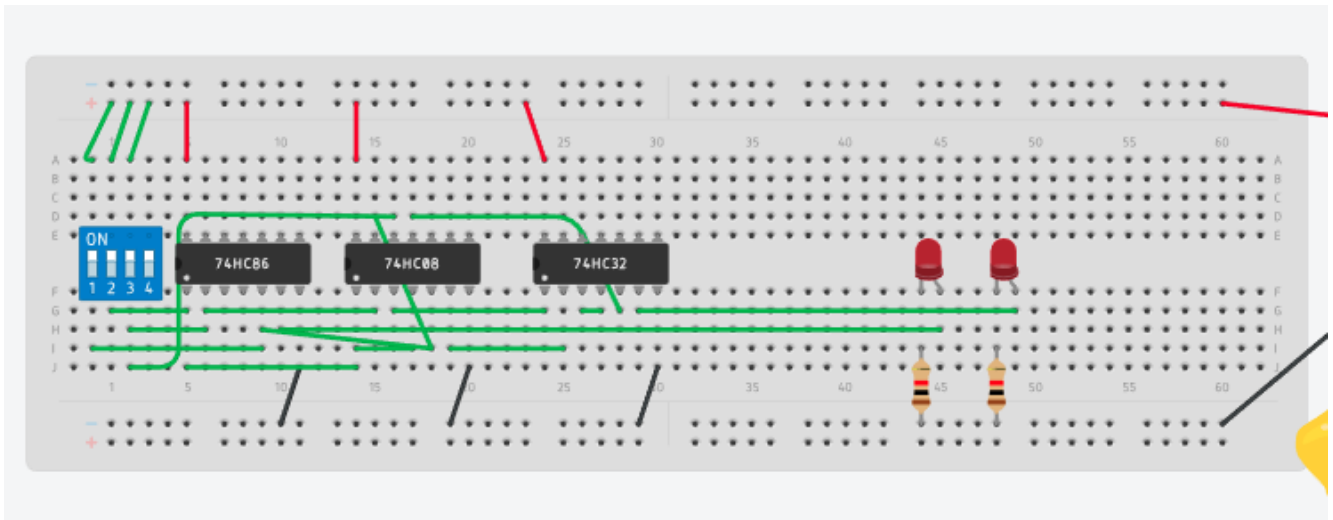


Figura 11 - Somador completo de 1 bit montado no Tinkercad.

Parte 6 - Somador completo no Arduino

O programa em C++ implementa as portas AND, OR e XOR como funções e usa LEDs para indicar Soma e Carryout.

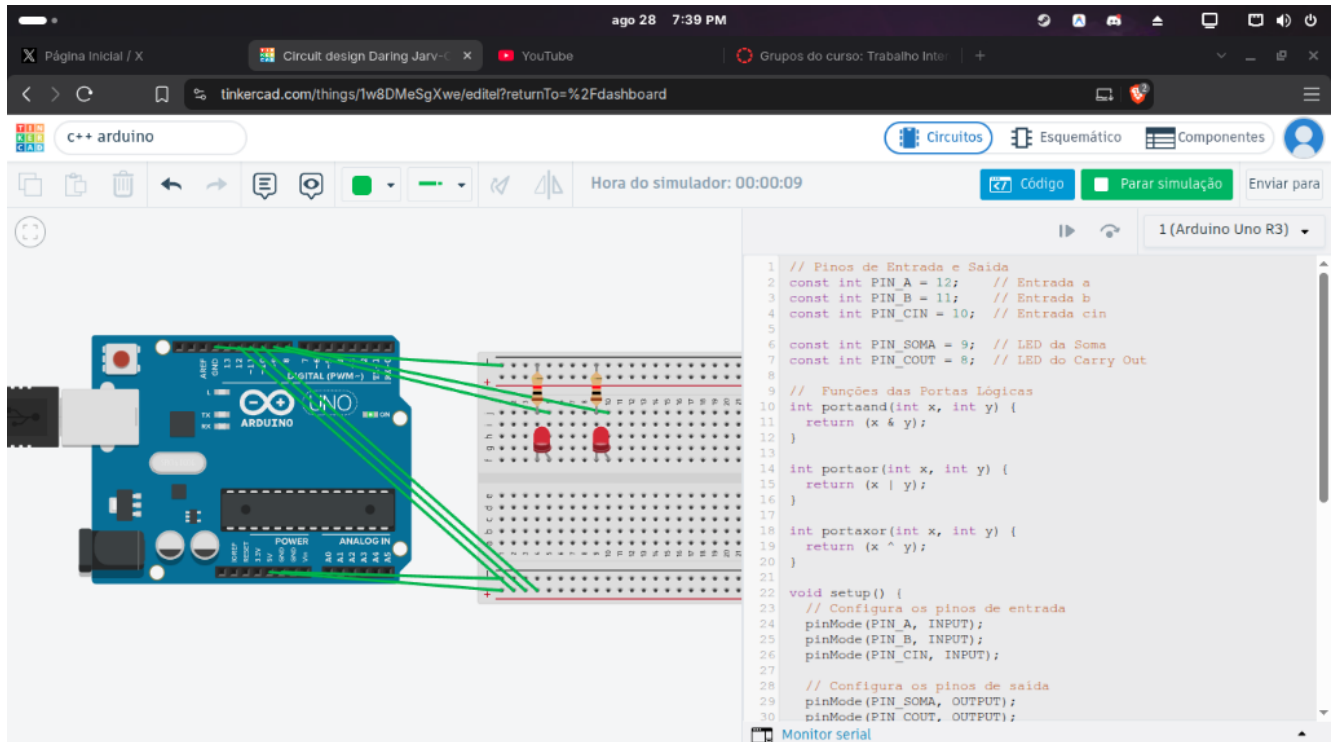


Figura 12 - Simulação do Arduino e trecho do programa elaborado.

Revisão antes de entregar: completar identificação, conferir se a captura do código mostra todo o programa e confirmar com o professor a interpretação das operações que usam ± 9 em 4 bits com sinal.